

Structured Query Language

SQL

Table of Contents	
Module 1 : RDBMS, Schema, Table Concepts & Data Types	
Module 2 : SELECT, WHERE, ORDER BY	
Module 3 : GROUP BY, HAVING	
Module 4 : INNER, LEFT, RIGHT, and FULL JOIN	
Module 5 : Subqueries (Scalar, Correlated, In SELECT/WHERE/FROM)	
Module 6 : Window Functions: OVER(), ROW_NUMBER(), RANK(), LAG(), LEAD()	
Module 7 : CASE WHEN, COALESCE, Date Functions	
Module 8 : Data Cleaning: TRIM, REPLACE, Removing Duplicates	

Module 1: RDBMS, Schema, Table Concepts & Data Types

1.1 What is an RDBMS?

A Relational Database Management System (RDBMS) stores data in tables made up of rows and columns. Each table represents one entity (e.g. Employees, Departments), and relationships between tables are created using keys. Common RDBMS products include MySQL, PostgreSQL, SQL Server, and Oracle. This course uses standard SQL syntax that works across most of these systems, with notes where a system differs.

1.2 Databases, Schemas and Tables

- Database: a container that holds related tables, views, and other objects.
- Schema: a logical grouping of tables inside a database (e.g. 'hr', 'sales'). In MySQL, 'schema' and 'database' are often used interchangeably.
- Table: a structured collection of rows (records) and columns (fields).
- Primary Key: a column (or set of columns) that uniquely identifies each row.
- Foreign Key: a column that references the primary key of another table, creating a relationship.

1.3 Common Data Types

Category	Data Type	Typical Use
Integer	INT / BIGINT	IDs, counts, whole numbers
Decimal	DECIMAL(p,s) / NUMERIC	Money, precise decimals
Floating point	FLOAT / DOUBLE	Approximate scientific values
Text (fixed)	CHAR(n)	Fixed-length codes
Text (variable)	VARCHAR(n)	Names, emails, descriptions
Large text	TEXT	Long free-form text
Date	DATE	Calendar date (YYYY-MM-DD)
Date & time	DATETIME / TIMESTAMP	Date plus time of day
Boolean	BOOLEAN / BIT	True/false flags

1.4 Creating the Practice Database

All examples in this document use one running database called company_db, with three related tables: departments, employees, and sales. We build it once here and reuse it in every module below.

Step 1: Create the database

```
CREATE DATABASE company_db;
USE company_db;
```

Step 2: Create the departments table

```
CREATE TABLE departments (
  department_id INT PRIMARY KEY,
  department_name VARCHAR(50) NOT NULL,
  location VARCHAR(50)
);
```

Step 3: Create the employees table

```
CREATE TABLE employees (  
    employee_id      INT PRIMARY KEY,  
    first_name       VARCHAR(50),  
    last_name        VARCHAR(50),  
    email            VARCHAR(100),  
    hire_date        DATE,  
    salary           DECIMAL(10,2),  
    department_id    INT,  
    manager_id       INT,  
    FOREIGN KEY (department_id) REFERENCES departments(department_id)  
);
```

Step 4: Create the sales table

```
CREATE TABLE sales (  
    sale_id          INT PRIMARY KEY,  
    employee_id      INT,  
    sale_date        DATE,  
    product_name     VARCHAR(50),  
    amount           DECIMAL(10,2),  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)  
);
```

Step 5: Insert sample data

```
INSERT INTO departments VALUES  
(1, 'Sales', 'Bengaluru'),  
(2, 'Marketing', 'Mumbai'),  
(3, 'IT', 'Hyderabad');  
  
INSERT INTO employees VALUES  
(101, 'Anil', 'Kumar', 'anil.kumar@corp.com', '2019-03-14', 55000, 1, NULL),  
(102, 'Divya', 'Rao', 'divya.rao@corp.com', '2020-07-01', 48000, 1, 101),  
(103, 'Rahul', 'Shah', 'rahul.shah@corp.com', '2018-01-22', 62000, 2, NULL),  
(104, 'Meera', 'Nair', 'meera.nair@corp.com', '2021-11-05', 45000, 2, 103),  
(105, 'Sanjay', 'Verma', 'sanjay.verma@corp.com', '2022-05-19', 51000, 3, NULL),  
(106, 'Priya', 'Iyer', NULL, '2023-02-10', 47000, 3, 105);  
  
INSERT INTO sales VALUES  
(1001, 101, '2024-01-05', 'Laptop', 75000.00),  
(1002, 102, '2024-01-08', 'Mouse', 500.00),  
(1003, 103, '2024-01-10', 'Tablet', 32000.00),  
(1004, 104, '2024-01-12', 'Monitor', 12000.00),  
(1005, 101, '2024-02-01', 'Laptop', 78000.00),  
(1006, 105, '2024-02-03', 'Keyboard', 1500.00),  
(1007, 106, '2024-02-14', 'Monitor', 11500.00),  
(1008, 102, '2024-03-01', 'Laptop', 76000.00);
```

Note: Every query example in this document runs against these three tables, so you can build the database once and follow along topic by topic.

Module 2: SELECT, WHERE, ORDER BY

2.1 SELECT - retrieving columns

SELECT chooses which columns to display. Use * to select all columns, or list specific column names.

```
-- All columns
SELECT * FROM employees;

-- Specific columns
SELECT first_name, last_name, salary
FROM employees;
```

2.2 WHERE - filtering rows

WHERE filters rows before they are returned, based on a condition.

```
-- Employees earning more than 50000
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 50000;

-- Employees in department 1, hired after 2019
SELECT * FROM employees
WHERE department_id = 1
  AND hire_date > '2019-12-31';

-- Pattern match: names starting with 'A'
SELECT first_name FROM employees
WHERE first_name LIKE 'A%';

-- Multiple allowed values
SELECT * FROM sales
WHERE product_name IN ('Laptop', 'Tablet');
```

2.3 ORDER BY - sorting results

ORDER BY sorts the result set. Default order is ascending (ASC); use DESC for descending.

```
-- Highest paid employees first
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC;

-- Sort by department, then salary within department
SELECT department_id, first_name, salary
FROM employees
ORDER BY department_id ASC, salary DESC;
```

Module 3: GROUP BY, HAVING

3.1 GROUP BY - aggregating rows

GROUP BY collapses rows sharing the same value into a single summary row, typically combined with aggregate functions like COUNT, SUM, AVG, MIN, MAX.

```
-- Total salary cost and headcount per department
SELECT department_id,
       COUNT(*)      AS employee_count,
       SUM(salary)    AS total_salary,
       AVG(salary)    AS avg_salary
FROM employees
GROUP BY department_id;

-- Total sales amount per product
SELECT product_name, SUM(amount) AS total_revenue
FROM sales
GROUP BY product_name
ORDER BY total_revenue DESC;
```

3.2 HAVING - filtering aggregated groups

WHERE filters rows before grouping; HAVING filters groups after aggregation. You cannot use an aggregate function inside WHERE.

```
-- Departments whose average salary exceeds 48000
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 48000;

-- Products with total revenue above 20000
SELECT product_name, SUM(amount) AS total_revenue
FROM sales
GROUP BY product_name
HAVING SUM(amount) > 20000;
```

Note: Order of execution: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Module 4: INNER, LEFT, RIGHT, and FULL JOIN

Joins combine rows from two or more tables based on a related column, usually a foreign key relationship. All examples below join employees to departments (employees.department_id = departments.department_id).

4.1 INNER JOIN

Returns only rows that have a match in both tables.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
INNER JOIN departments d
      ON e.department_id = d.department_id;
```

4.2 LEFT JOIN

Returns all rows from the left table, and matching rows from the right table. Unmatched right-side columns appear as NULL.

```
-- All employees, with department name if one exists
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d
      ON e.department_id = d.department_id;

-- Find employees with no recorded sales at all
SELECT e.employee_id, e.first_name
FROM employees e
LEFT JOIN sales s
      ON e.employee_id = s.employee_id
WHERE s.sale_id IS NULL;
```

4.3 RIGHT JOIN

The mirror image of LEFT JOIN: returns all rows from the right table, and matching rows from the left table.

```
-- All departments, even ones with no employees yet
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d
      ON e.department_id = d.department_id;
```

4.4 FULL (OUTER) JOIN

Returns all rows from both tables, matching where possible and filling unmatched sides with NULL.

```
SELECT e.first_name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
      ON e.department_id = d.department_id;

-- MySQL has no FULL JOIN keyword; simulate with UNION
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id
UNION
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id;
```

Module 5: Subqueries

A subquery is a query nested inside another query. It can appear in the SELECT list, the WHERE clause, or the FROM clause.

5.1 Scalar Subquery

Returns a single value, and can be used anywhere a single value is expected.

```
-- Show each employee's salary next to the company-wide average
SELECT first_name, salary,
       (SELECT AVG(salary) FROM employees) AS company_avg_salary
FROM employees;
```

5.2 Subquery in WHERE

Commonly used with IN, =, >, or comparison operators to filter based on a computed set or value.

```
-- Employees who earn more than the average salary
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Employees who have made at least one sale
SELECT first_name, last_name
FROM employees
WHERE employee_id IN (SELECT DISTINCT employee_id FROM sales);
```

5.3 Correlated Subquery

A correlated subquery references a column from the outer query, so it re-runs once per outer row (unlike a plain subquery, which runs once).

```
-- Employees earning more than the average salary in their OWN department
SELECT e1.first_name, e1.department_id, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department_id = e1.department_id
);
```

5.4 Subquery in FROM (Derived Table)

A subquery in the FROM clause is treated as a temporary, in-memory table and must be given an alias.

```
-- Rank departments by total revenue using a derived table
SELECT dept_totals.department_id, dept_totals.total_revenue
FROM (
    SELECT e.department_id, SUM(s.amount) AS total_revenue
    FROM employees e
    JOIN sales s ON e.employee_id = s.employee_id
    GROUP BY e.department_id
) AS dept_totals
ORDER BY dept_totals.total_revenue DESC;
```

Module 6: Window Functions

Window functions perform calculations across a set of rows related to the current row, without collapsing the rows the way GROUP BY does. Every window function uses the OVER() clause, optionally with PARTITION BY (grouping) and ORDER BY (row order within the group).

6.1 OVER() and PARTITION BY

- OVER() is a window function clause that defines the set of rows (called a *window*) on which a window function performs its calculation. Unlike GROUP BY, it does not combine rows into a single result; instead, it returns every row while adding the calculated value.
- PARTITION BY is an optional clause used inside OVER() to divide the result set into separate groups (partitions). The window function is then calculated independently for each group, while still displaying every row.

```
-- Running total of sales amount per employee, ordered by date
SELECT employee_id, sale_date, amount,
       SUM(amount) OVER (
         PARTITION BY employee_id
         ORDER BY sale_date
       ) AS running_total
FROM sales;
```

6.2 ROW_NUMBER()

Assigns a unique, sequential number to each row within its partition, with no ties.

```
-- Number each employee's sales in date order
SELECT employee_id, sale_date, amount,
       ROW_NUMBER() OVER (
         PARTITION BY employee_id
         ORDER BY sale_date
       ) AS sale_sequence
FROM sales;
```

6.3 RANK() and DENSE_RANK()

RANK() assigns the same rank to tied rows and skips subsequent numbers; DENSE_RANK() also ties but does not skip.

```
-- Rank employees by salary, highest first
SELECT first_name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_dense_rank
FROM employees;
```

6.4 LAG() and LEAD()

LAG() looks at a previous row's value; LEAD() looks at a following row's value, both within the same ordered partition.

```
-- Compare each sale to the employee's previous sale
SELECT employee_id, sale_date, amount,
       LAG(amount, 1) OVER (
         PARTITION BY employee_id ORDER BY sale_date
       ) AS previous_sale_amount,
       LEAD(amount, 1) OVER (
         PARTITION BY employee_id ORDER BY sale_date
       ) AS next_sale_amount
FROM sales;
```

Module 7: CASE WHEN, COALESCE, Date Functions

7.1 CASE WHEN - conditional logic

CASE WHEN evaluates conditions in order and returns the value for the first one that is true, similar to if/else logic.

```
-- Classify employees into salary bands
SELECT first_name, salary,
       CASE
         WHEN salary >= 60000 THEN 'High'
         WHEN salary >= 48000 THEN 'Medium'
         ELSE 'Entry'
       END AS salary_band
FROM employees;
```

7.2 COALESCE - handling NULLs

COALESCE returns the first non-NULL value from a list of expressions, useful for supplying default values.

```
-- Show 'No Email' instead of NULL
SELECT first_name,
       COALESCE(email, 'No Email') AS contact_email
FROM employees;

-- Fallback chain: prefer manager, else 'Top Level'
SELECT first_name,
       COALESCE(manager_id, 0) AS manager_or_zero
FROM employees;
```

7.3 Date Functions: DATEDIFF and DATEADD

DATEDIFF calculates the difference between two dates; DATEADD (or DATE_ADD in MySQL) shifts a date forward or backward by an interval.

```
-- SQL Server style
-- Days each employee has been with the company
SELECT first_name, hire_date,
       DATEDIFF(DAY, hire_date, GETDATE()) AS days_employed
FROM employees;

-- Add 90 days to hire_date to find end of probation
SELECT first_name, hire_date,
       DATEADD(DAY, 90, hire_date) AS probation_end_date
FROM employees;

-- MySQL equivalents
SELECT first_name, hire_date,
       DATEDIFF(CURDATE(), hire_date) AS days_employed
FROM employees;

SELECT first_name, hire_date,
       DATE_ADD(hire_date, INTERVAL 90 DAY) AS probation_end_date
FROM employees;
```

Module 8: Data Cleaning

Real-world data is rarely perfect. This module covers trimming whitespace, replacing unwanted characters, and removing duplicate rows.

8.1 TRIM - removing extra spaces

```
-- Remove leading/trailing spaces from names
SELECT TRIM(first_name) AS cleaned_first_name
FROM employees;

-- LTRIM / RTRIM for one side only
SELECT LTRIM(first_name) AS left_trimmed,
       RTRIM(first_name) AS right_trimmed
FROM employees;
```

8.2 REPLACE - substituting text

```
-- Standardise email domain
SELECT email,
       REPLACE(email, '@corp.com', '@newcorp.com') AS updated_email
FROM employees
WHERE email IS NOT NULL;
```

8.3 Removing Duplicate Rows

DISTINCT removes duplicate rows from a result set. For deleting duplicates permanently, a common pattern uses ROW_NUMBER() to keep only the first occurrence.

```
-- View distinct product names sold
SELECT DISTINCT product_name FROM sales;

-- Identify duplicate rows (same employee, same sale_date, same amount)
SELECT employee_id, sale_date, amount, COUNT(*) AS occurrences
FROM sales
GROUP BY employee_id, sale_date, amount
HAVING COUNT(*) > 1;

-- Delete duplicates, keeping only the lowest sale_id per group
WITH ranked_sales AS (
    SELECT sale_id,
           ROW_NUMBER() OVER (
               PARTITION BY employee_id, sale_date, amount
               ORDER BY sale_id
           ) AS rn
    FROM sales
)
DELETE FROM sales
WHERE sale_id IN (SELECT sale_id FROM ranked_sales WHERE rn > 1);
```

Note: Always run the SELECT/identification query first and review the rows before running a DELETE, so you don't remove data you meant to keep.